

PROGRAM 1

Write a PL/SQL block to calculate the incentive of an employee whose ID is 110.

DECLARE

```
v_empid employees.employee_id %TYPE := 110;  
v_salary employees.salary %TYPE;  
v_incentive NUMBER;
```

BEGIN

```
SELECT salary INTO v_salary  
FROM employees
```

```
WHERE employee_id = v_empid;
```

```
IF v_salary >= 10000 THEN  
    v_incentive := v_salary * 0.10;
```

```
ELSE  
    v_incentive := v_salary * 0.05;
```

```
END IF;
```

```
DBMS_OUTPUT.PUT_LINE ('Employee ID: ' || v_empid);
```

```
DBMS_OUTPUT.PUT_LINE ('Salary: ' || v_salary);
```

```
DBMS_OUTPUT.PUT_LINE ('Incentive: ' || v_incentive);
```

```
END;
```

PROGRAM 2

Write a PL/SQL block to show an invalid case-insensitive reference to a quoted and without quoted user-defined identifier.

DECLARE

'My Variable' NUMBER := 100;

BEGIN

DBMS-OUTPUT.PUT_LINE ('My Variable'); -- End

END;

corrected;

DBMS-OUTPUT.PUT_LINE ('"My Variable"');

PROGRAM 3

Write a PL/SQL block to adjust the salary of the employee whose ID 122.

Sample table: employees

DECLARE:

v_emp_id employees.employee_id %TYPE := 122;

v_salary employees.salary %TYPE;

BEGIN

SELECT salary INTO v_salary FROM employees WHERE
employee_id = v_emp_id;

v_salary := v_salary + (v_salary * 0.10);

UPDATE employees

SET salary = v_salary.

WHERE employee_id = v_emp_id;

DBMS_OUTPUT.PUT_LINE ('Salary updated for employee ' || v_emp_id);

END;

/

PROGRAM 4

Write a PL/SQL block to create a procedure using the "IS [NOT] NULL Operator" and show AND operator returns TRUE if and only if both operands are TRUE.

~~DECLARE~~

V. ~~bonus~~ NUMBER := NULL;

V. Salary NUMBER := 5000;

BEGIN

IF V. ~~bonus~~ IS NULL THEN

DBMS_OUTPUT.PUT_LINE ('Bonus is NULL');
AND IF;

IF (V. salary > 3000) AND (V. salary < 6000) THEN
DBMS_OUTPUT.PUT_LINE ('Condition TRUE: salary between
3000 and 6000');

ELSE

DBMS_OUTPUT.PUT_LINE ('condition FALSE');

END IF;

END;

/

PROGRAM 5

Write a PL/SQL block to describe the usage of LIKE operator including wildcard characters and escape character.

```
DECLARE
v_name employees.last_name%TYPE;
BEGIN
FOR sec IN
    SELECT last_name FROM employees
    WHERE last_name LIKE 'S%' ESCAPE '\';
) LOOP
    DBMS_OUTPUT.PUT_LINE / 'Name starting with S: ' || sec.
                           (v_name);
END LOOP;
END;
```

PROGRAM 6

Write a PL/SQL program to arrange the number of two variable in such a way that the small number will store in num_small variable and large number will store in num_large variable.

DECLARE

num, NUMBER := 40;

num, NUMBER := 20;

num . small NUMBER;

num . large NUMBER;

BEGIN

IF num < num 2 THEN

num - small := num 1;

num - large := num 2;

ELSE

num - small := num 2;

num - large := num 1;

END IF;

DBMS_OUTPUT.PUT_LINE ('Small Number: ' || num small);

DBMS_OUTPUT.PUT_LINE ('Large Number: ' || num large);

END;

/

PROGRAM 7

Write a PL/SQL procedure to calculate the incentive on a target achieved and display the message either the record updated or not.

```
CREATE OR REPLACE PROCEDURE Calc_incentive (p_empid NUMBER),
```

```
  v_salary employees.salary % TYPE;
```

```
  v_incentive NUMBER;
```

```
BEGIN:
```

```
SELECT salary INTO v_salary FROM employees WHERE employee_id
```

```
= p_empid;
```

```
v_incentive := v_salary * 0.10;
```

```
UPDATE employees
```

```
SET commission_pct = NVL (commission_pct, 0) + v_incentive / 100
```

```
WHERE employee_id = p_empid;
```

```
IF SQL%ROWCOUNT > 0 THEN
```

```
  DBMS_OUTPUT.PUT_LINE ('Record updated successfully');
```

```
ELSE
```

```
  DBMS_OUTPUT.PUT_LINE ('NO Record Found');
```

```
END IF;
```

```
END;
```

```
/
```

```
BEGIN
```

```
  Calc_incentive (110);
```

```
END;
```

```
/
```


PROGRAM 8

Write a PL/SQL procedure to calculate incentive achieved according to the specific sale limit.

```

CREATE OR REPLACE PROCEDURE 'Sale-Incentive' (p-sale NUMBER) IS
    v_incentive NUMBER;

BEGIN
    IF p.sale >= 1000 THEN
        v_incentive := 0.20;
    ELSE IF p.sale >= 5000 THEN
        v_incentive := 0.10;
    ELSE
        v_incentive := 0.05;
    END IF;

    DBMS_OUTPUT.PUT_LINE ('Incentive Rate: ' || (v_incentive * 100)
                           || '%');

END;
/

```


PROGRAM 9

Write a PL/SQL program to count number of employees in department 50 and check whether this department have any vacancies or not. There are 45 vacancies in this department.

DECLARE

V-COUNT NUMBER;

V-VACANCIES CONSTANT NUMBER := 45;

BEGIN

SELECT COUNT (*) INTO V-COUNT FROM employees WHERE
department_id = 50;

DBMS-OUTPUT.PUT_LINE ('Employees in Dept 50: ' || V-COUNT);

~~IF~~ IF V-COUNT < V-VACANCIES THEN

DBMS-OUTPUT.PUT_LINE ('Vacancies available : ' ||

~~ELSE~~

(V-VACANCIES - V-COUNT));

DBMS-OUTPUT.PUT_LINE ('NO Vacancies.');

END IF;

END;

PROGRAM 10

Write a PL/SQL program to count number of employees in a specific department and check whether this department have any vacancies or not. If any vacancies, how many vacancies are in that department.

```

DECLARE
    v_dept NUMBER := &dept_id;
    v_count NUMBER;
    v_vacancies NUMBER := 45;
BEGIN
    SELECT COUNT(*) INTO v_count FROM employees WHERE
    department_id = v_dept;
    IF v_count < v_vacancies THEN
        DBMS_OUTPUT.PUT_LINE ('Dept' || v_dept || ' has ' ||
        (v_vacancies - v_count) ||
        'vacancies');
    ELSE
        DBMS_OUTPUT.PUT_LINE ('Dept' || v_dept || ' has no
        vacancies');
    END IF;
END;
/

```

PROGRAM 11

Write a PL/SQL program to display the employee IDs, names, job titles, hire dates, and salaries of all employees.

BEGIN

```
FOR emp-rec IN (SELECT employee-id, first_name || " " || last_name  
                AS full_name, job-id, hire-date, salary  
                FROM employees)
```

LOOP

```
DBMS-OUTPUT-LINE ('ID' || emp-rec.employee-id ||  
                  ', Name: ' || emp-rec.full_name ||  
                  ', Job: ' || emp-rec.job-id ||  
                  ', Hire Date: ' || emp-rec.hire-date ||  
                  ', salary: ' || emp-rec.salary);
```

END LOOP;

END;

/

PROGRAM 12

Write a PL/SQL program to display the employee IDs, names, and department names of all employees.

BEGIN

```
FOR emp_rec IN (SELECT e.employee_id, e.first_name || e.last_name
                    AS full_name, d.department_name
FROM employee e
JOIN department d ON e.department_id =
                    d.department_id)
```

LOOP

```
DBMS_OUTPUT.PUT_LINE('ID: ' || emp_rec.employee_id ||
                      ', Name: ' || emp_rec.full_name ||
                      ', Department: ' || emp_rec.department_name);
```

END LOOP;

END;

/

PROGRAM 13

Write a PL/SQL program to display the job IDs, titles, and minimum salaries of all jobs.

BEGIN

FOR job_rec (SELECT job_id, job_title, min_salary
FROM jobs)

LOOP

DBMS_OUTPUT.PUT_LINE ('Job-ID: ' || job_rec.job_id
, title: ' || job_rec.job_title
, Min salary: ' || job_rec.job_salary);

END LOOP;

END;

/

PROGRAM 14

Write a PL/SQL program to display the employee IDs, names, and job history start dates of all employees.

BEGIN

FOR hist_rec IN (SELECT e.employee_id, e.first_name || ' ' || e.last_name AS full_name, h.start_date

FROM employee e

JOIN job_history h ON e.employee_id = h.employee_id)

LOOP

DBMS_OUTPUT.PUT_LINE ('ID: ' || hist_rec.employee_id ||

, Name: ' || hist_rec.full_name ||

, job history start: ' || hist_rec.start_date);

END LOOP;

END;

/

PROGRAM 15

Write a PL/SQL program to display the employee IDs, names, and job history end dates of all employees.

```

BEGIN
FOR LIST_REC IN CURSOR (
    SELECT e.employee_id, p.first_name,
           h.job_history_id, h.end_date
    FROM employees e
    JOIN job_history h ON e.employee_id
                        = h.employee_id)
LOOP
    DBMS_OUTPUT.PUT_LINE ('Employee Id' || LIST_REC.employee_id ||
                           ' First Name' || LIST_REC.first_name ||
                           ' Job History Id' || LIST_REC.job_history_id ||
                           ' End Date' || LIST_REC.end_date);
END LOOP;

```

END;

/

Evaluation Procedure	Marks awarded
PL/SQL Procedure(5)	5
Program/Execution (5)	5
Viva(5)	5
Total (15)	15
Faculty Signature	